

L1-09 群时延 All-Pass 补偿算法行为级仿真报告

L1-09 群时延 All-Pass 补偿算法行为级仿真报告

摘要

本文针对 L1-09 群时延补偿算法进行行为级仿真验证。L1-09 的目标是在 L1-08 完成幅频补偿后，继续针对链路中的 phase / group delay distortion 进行补偿，使系统在目标频带内的群时延更加平坦。当前实现采用多级二阶 all-pass IIR filter 作为补偿结构。该结构理论上不改变幅度响应，只改变相位响应，因此适合接在 L1-08 幅频 FIR 补偿之后，用于独立改善 phase / group delay 问题。

当前 pipeline 中，L1-09 的输入不是原始 H1，而是已经经过 L1-08 fixed-point FIR 后的响应：

$$\text{pre_L1_09_response}(f) = H_1(f) H_{2,\text{fixed}}(f)$$

其中 H1 表示模拟硬件链路的复数频率响应，H2_fixed 表示 L1-08 fixed-point FIR 补偿器。L1-09 首先读取该复数响应中的相位，执行 unwrap 后计算 group delay，再通过 least-squares 优化多级二阶 all-pass filter 的 pole radius 和 pole angle，使补偿后的 group delay 尽量接近常数。

在当前 clean full pipeline run full_combined_20260605_155348 中，L1-09 前的 group delay ripple 为 2.994724 ns；floating-point all-pass 补偿后降为 2.108406 ns；fixed-point all-pass 补偿后为 2.108794 ns。因此当前 L1-09 已经降低 group delay ripple，但尚未完全拉平。fixed-point all-pass 的 pole 均位于单位圆内，stable=True，且 saturation_count=0，说明当前系数量化后滤波器数值稳定。

git repo: <https://github.com/Yuzhe2005/L1-08-L1-09>

1. 术语与缩略词

术语	含义
L1-08	幅频 FIR 补偿算法，主要补偿 magnitude ripple
L1-09	相位 / 群时延补偿算法，主要补偿 group delay ripple

术语	含义
H1	模拟硬件链路的复数频率响应
H2	L1-08 FIR 补偿器的频率响应
H2_fixed	fixed-point 量化后的 L1-08 FIR 频率响应
all-pass filter	幅度恒为 1、只改变相位的滤波器
IIR	Infinite Impulse Response, 无限冲激响应滤波器
SOS	Second-Order Section, 二阶滤波器 section
phase	相位响应, 单位 rad
unwrap	去除 phase 中的 2 跳变, 使相位曲线连续
group delay	群时延, 表示不同频率分量的传播延迟
pole	IIR filter 分母多项式的根, 决定反馈系统稳定性
fixed-point	固定 bit 宽的定点数表示
saturation	数值超过 fixed-point 可表示范围后被截断
EVM	Error Vector Magnitude, 误差向量幅度
EVM_LIN	基于频率响应的线性 EVM 辅助指标
QAM	Quadrature Amplitude Modulation, 正交幅度调制

2. 问题背景与算法目标

2.1 L1-09 需要解决的问题

真实硬件链路不仅会造成幅度不平坦, 也会造成相位非线性。相位非线性直接结果是 group delay 不平坦。对于宽带信号, 不同频率分量如果经历不同延迟, 最终叠加回时域时会发生波形畸变, 从而影响调制信号质量。

L1-08 已经负责补偿幅频不平坦问题:

$$|H_1(f) H_{2,\text{fixed}}(f)| \text{ 尽量平坦}$$

但是 L1-08 当前使用的是 real linear-phase FIR。它本身只引入近似常数 group delay, 不负责主动修复 H1 中非线性相位造成的 group delay ripple。因此 L1-09 需要在 L1-08 之后继续处理:

phase / group delay distortion

更准确地说, L1-09 当前分析和补偿的是:

$$\text{pre_L1_09_response}(f) = H_1(f) H_{2,\text{fixed}}(f)$$

也就是信号已经通过 L1-08 fixed-point FIR 后，准备进入 L1-09 模块之前的复数响应。

2.2 算法目标

L1-09 当前行为级仿真的目标包括：

1. 从 $H_1 * H_{2,\text{fixed}}$ 的复数响应中提取相位。
2. 对相位执行 `unwrap`，得到连续相位曲线。
3. 根据连续相位计算 group delay。
4. 使用多级二阶 all-pass IIR filter 产生额外的频率相关延迟。
5. 使补偿后的 group delay 尽量接近常数。
6. 对 all-pass 系数进行 fixed-point 量化。
7. 检查量化后 IIR filter 的 pole 是否位于单位圆内。
8. 使用 EVM_LIN 和 QAM EVM 验证补偿前后的信号质量变化。

当前阶段的重点是行为级算法验证，不是完整 RTL 级 fixed-point 仿真。也就是说，当前 fixed-point 主要量化 all-pass 系数并重新计算频率响应，尚未完整模拟每一级 IIR 运算中的乘法器、加法器、累加器、寄存器和逐级 rounding / saturation。

3. 理论模型

3.1 复数频率响应

硬件链路 H1 可以写成：

$$H_1(f) = |H_1(f)| \exp(j\phi_{H_1}(f))$$

其中：

$$\begin{aligned} |H_1(f)| &\text{ 表示 magnitude response,} \\ \phi_{H_1}(f) &\text{ 表示 phase response.} \end{aligned}$$

经过 L1-08 fixed-point FIR 后，进入 L1-09 之前的响应为：

$$H_{\text{pre}}(f) = H_1(f) H_{2,\text{fixed}}(f)$$

L1-09 关心的是 $H_{\text{pre}}(f)$ 的 phase 和 group delay。

3.2 Phase Unwrap

计算机中直接从复数响应得到的 phase 通常来自：

$$\angle H_{\text{pre}}(f)$$

这个 phase 会被限制在：

$$[-\pi, \pi]$$

因此真实相位如果连续下降或上升超过 π ，就会在图上出现突然跳变。unwrap 的作用是把这些人为了 2π 跳变去掉，让相位变成连续曲线。

例如 wrapped phase 可能类似：

2.9, 3.1, -3.0, -2.8

unwrap 后会变成：

2.9, 3.1, 3.283, 3.483

unwrap 不改变真实物理相位，只是把相位表示方式从“折叠形式”改成“连续形式”。这是计算 group delay 之前必须做的步骤，避免对信号跳变求导。

3.3 Group Delay 定义

group delay 定义为相位对角频率的负导数：

$$\tau_g(f) = -\frac{d\phi(f)}{d\omega}$$

其中：

$$\omega = 2\pi f$$

如果相位是严格线性的：

$$\phi(f) = -2\pi f\tau$$

则 group delay 为常数：

$$\tau_g(f) = \tau$$

这表示所有频率分量经历同样的延迟，信号形状不会因为不同频率到达时间不同而被拉扯。反过来，如果 group delay 随频率起伏，就说明不同频率分量经历不同延迟，会造成宽带信号畸变。

3.4 All-Pass Filter 的作用

all-pass filter 的核心特点是：

$$|A(e^{j\Omega})| = 1$$

也就是说，它不改变幅度，只改变相位：

$$A(e^{j\Omega}) = \exp(j\phi_A(\Omega))$$

因此 all-pass filter 很适合接在 L1-08 后面：

L1-08：主要改变 **magnitude**

L1-09：主要改变 **phase / group delay**

L1-09 补偿后的响应可以写成：

$$H_{\text{out}}(f) = H_1(f) H_{2,\text{fixed}}(f) A(f)$$

由于 $|A(f)| = 1$ ，理论上：

$$|H_{\text{out}}(f)| = |H_1(f) H_{2,\text{fixed}}(f)|$$

所以 L1-09 不应该破坏 L1-08 已经完成的幅频补偿。

3.5 二阶 All-Pass Section

当前程序使用多级二阶 all-pass section。单个二阶 all-pass section 写成：

$$A_k(z) = \frac{r_k^2 - 2r_k \cos(\theta_k)z^{-1} + z^{-2}}{1 - 2r_k \cos(\theta_k)z^{-1} + r_k^2 z^{-2}}$$

其中：

r_k pole radius,

θ_k pole angle.

分母决定 pole：

$$1 - 2r_k \cos(\theta_k)z^{-1} + r_k^2 z^{-2} = 0$$

它对应一对共轭 pole：

$$z = r_k \exp(+j\theta_k),$$

$$z = r_k \exp(-j\theta_k).$$

所以 r_k 决定 pole 离单位圆多远, θ_k 决定该 section 主要作用在哪个频率附近。

3.6 Pole 与稳定性

IIR filter 有反馈项, 因此稳定性由 pole 决定。对于数字 IIR filter:

所有 pole 都在单位圆内 $\Rightarrow$ stable,
任意 pole 在单位圆外 $\Rightarrow$ unstable.

单位圆内表示:

$$|\text{pole}| < 1$$

对于当前二阶 all-pass section:

$$|\text{pole}| = r_k$$

因此只要:

$$r_k < 1$$

该 section 就是稳定的。当前 fixed-point 检查中的 `stable=True` 衡量的正是所有量化后的 all-pass IIR pole 是否仍然位于单位圆内。

3.7 多级 All-Pass Cascade

单个二阶 section 的补偿能力有限, 因此当前 L1-09 使用多个 section 串联:

$$A_{\text{total}}(z) = A_1(z)A_2(z) \cdots A_N(z)$$

当前 active 配置为:

$$N = 8$$

串联后的总 phase 为各级 phase 之和:

$$\phi_{A,\text{total}}(f) = \sum_k \phi_{A,k}(f)$$

总 group delay 也近似为各级 group delay 之和:

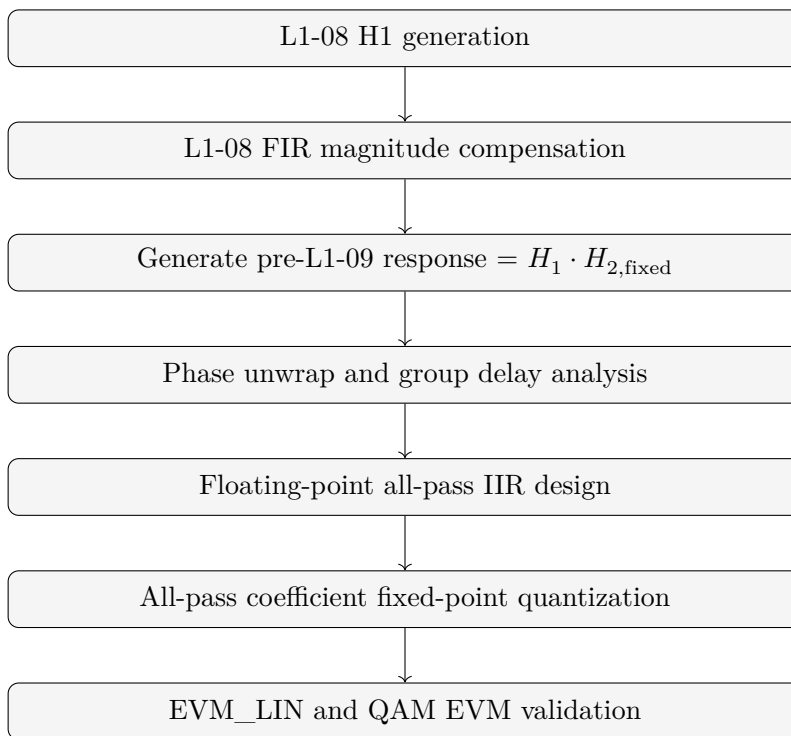
$$\tau_{A,\text{total}}(f) = \sum_k \tau_{A,k}(f)$$

这使得 L1-09 可以通过多个局部 all-pass section 共同拟合复杂的 group delay 补偿曲线。

4. 行为级仿真方案设计

4.1 总体流程

当前 L1-09 pipeline 位于完整 L1-08 + L1-09 pipeline 的后半段:



完整入口命令为:

```
.venv\Scripts\python.exe run_full_l1_08_l1_09_pipeline.py
```

L1-09 单独入口为:

```
.venv\Scripts\python.exe L1_09_sim\run_all_l1_09_pipeline.py --run-dir
↪ data\<run_folder>
```

4.2 L1-09 程序结构

文件	功能
L1_09_group_delay_analyzer.py	读取 H1 和 L1-08 fixed FIR 响应，计算 H1 * H2_fixed 的 phase 和 group delay
L1_09_allpass_designer.py	设计 floating-point 多级二阶 all-pass IIR
L1_09_fixed_point_quantizer.py	量化 all-pass 系数，检查稳定性并重新计算 fixed response
L1_09_evm_lin_calculator.py	基于频率响应计算 EVM_LIN、magnitude-only EVM 和 phase-only EVM
L1_09_qam_evm_validator.py	用 QAM-loaded IF 信号验证 L1-09 前后 EVM 变化
run_all_l1_09_pipeline.py	串联运行完整 L1-09 pipeline
L1_09_config.py	读取 L1-09 配置

4.3 数据输入与输出

当前项目目标输出规则为：

data/ 保存 csv、json 等数据文件
graph/ 保存 png 图像文件

一次 full pipeline 会生成：

data/full_combined_YYYYMMDD_HHMMSS/
graph/full_combined_YYYYMMDD_HHMMSS/

L1-09 主要输入包括：

data/<run>/h1_full_combined_random/together.csv
data/<run>/l1_08_h2_fixed_point/h2_fixed_point_response.csv

其中 together.csv 提供 H1 的 magnitude 和 phase，h2_fixed_point_response.csv 提供 L1-08 fixed FIR 的频率响应。

L1-09 主要输出包括：

data/<run>/l1_09_fix_group_delay/group_delay_analysis.csv
data/<run>/l1_09_fix_group_delay/group_delay_metrics.csv

graph/<run>/l1_09_fix_group_delay/phase_before_l1_09.png
graph/<run>/l1_09_fix_group_delay/group_delay_before_l1_09.png
graph/<run>/l1_09_fix_allpass_iir_fs/allpass_coefficients.csv


```

graph/<run>/l1_09_fix_allpass_iir_fs/allpass_response.csv
graph/<run>/l1_09_fix_allpass_iir_fs/allpass_metrics.csv
graph/<run>/l1_09_fix_allpass_iir_fs/group_delay_before_after_l1_09.png
graph/<run>/l1_09_fix_allpass_iir_fixed/allpass_coefficients_fixed.csv
graph/<run>/l1_09_fix_allpass_iir_fixed/allpass_fixed_response.csv
graph/<run>/l1_09_fix_allpass_iir_fixed/allpass_fixed_metrics.csv
graph/<run>/l1_09_fix_allpass_iir_fixed/allpass_fixed_quantization.png
graph/<run>/l1_09_fix_evm_lin_fixed/evm_lin.png
graph/<run>/l1_09_fix_qam_evm_iir_fixed/l1_09_qam_evm.png

```

5. All-Pass 设计方法

5.1 为什么不能直接“提前”某些频率

真实物理系统不能让信号提前到达，只能增加延迟。因此 L1-09 的设计逻辑不是让慢的频率变快，而是让快的频率多等一会，使所有频率分量尽量接近同一个更晚的目标延迟。

程序中 target delay 的基本思想是：

$$\text{target_delay_ns} = \max(\text{smoothed_group_delay_ns}) + \text{margin_ns}$$

其中 margin_ns 用于留出一点补偿余量。当前如果 config 中没有显式指定 margin，程序会使用：

$$\text{margin_ns} = \max(0.05 \text{ ns}, 5\% \cdot \text{original_ripple_ns})$$

当前 clean run 中：

$$\begin{aligned} \text{target_delay_ns} &= 9.813229332 \text{ ns}, \\ \text{target_margin_ns} &= 0.087111678 \text{ ns}. \end{aligned}$$

5.2 平滑处理

原始 group delay 中可能包含局部数值抖动。如果直接拟合所有尖锐抖动，all-pass filter 可能会过拟合，得到不够平滑或不够稳定的补偿结构。因此程序先使用 moving average 对 group delay 进行平滑：

$$\text{fit_delay_ns} = \text{moving_average}(\text{group_delay_ns}, \text{smooth_window})$$

当前配置：

$$\text{smooth_window} = 31$$

需要注意，平滑只用于优化拟合目标；最终评估 compensated group delay ripple 时仍然使用未平滑的原始 group delay 加上 all-pass group delay。

5.3 优化变量

当前每个二阶 section 有两个主要参数：

$$r_k, \quad \theta_k$$

对于 N 个二阶 section，优化变量为：

$$[r_1, r_2, \dots, r_N, \theta_1, \theta_2, \dots, \theta_N, \text{target_delay}]$$

当前 active 配置：

$$N = 8$$

因此需要优化 17 个量：

$$8 \text{ 个 } r, \quad 8 \text{ 个 } \theta, \quad 1 \text{ 个 } \text{target_delay}$$

5.4 Least-Squares 目标函数

all-pass 产生的 group delay 记为：

$$\tau_A(f)$$

补偿后的 group delay 为：

$$\tau_{\text{comp}}(f) = \tau_{\text{pre}}(f) + \tau_A(f)$$

优化目标是让它尽量接近常数 **target_delay**：

$$\min \sum_f [\tau_{\text{pre,smooth}}(f) + \tau_A(f) - \text{target_delay}]^2$$

程序中为了数值稳定，会除以一个 scale：

$$\text{residual}(f) = \frac{\tau_{\text{pre,smooth}}(f) + \tau_A(f) - \text{target_delay}}{\max(\text{original_ripple_ns}, 1.0)}$$

该 scale 不改变优化目标的物理含义，只是让优化器中的数值量级更舒服。

5.5 当前 All-Pass 设计结果

当前 clean run 为：

full_combined_20260605_155348

主要 all-pass 设计结果如下：

指标	数值
sampling rate	12 GHz
all-pass section count	8
smooth window	31
original group delay ripple	2.994724 ns
floating compensated group delay ripple	2.108406 ns
compensated RMS to target	0.364910 ns
optimizer success	True

代表性图像如下：



6. Fixed-Point 量化

6.1 当前系数量化格式

当前 L1-09 all-pass filter 的系数量化格式为：

signed fixed-point Q3.15

对应：

total_bits = 18,
frac_bits = 15.

量化步进为：

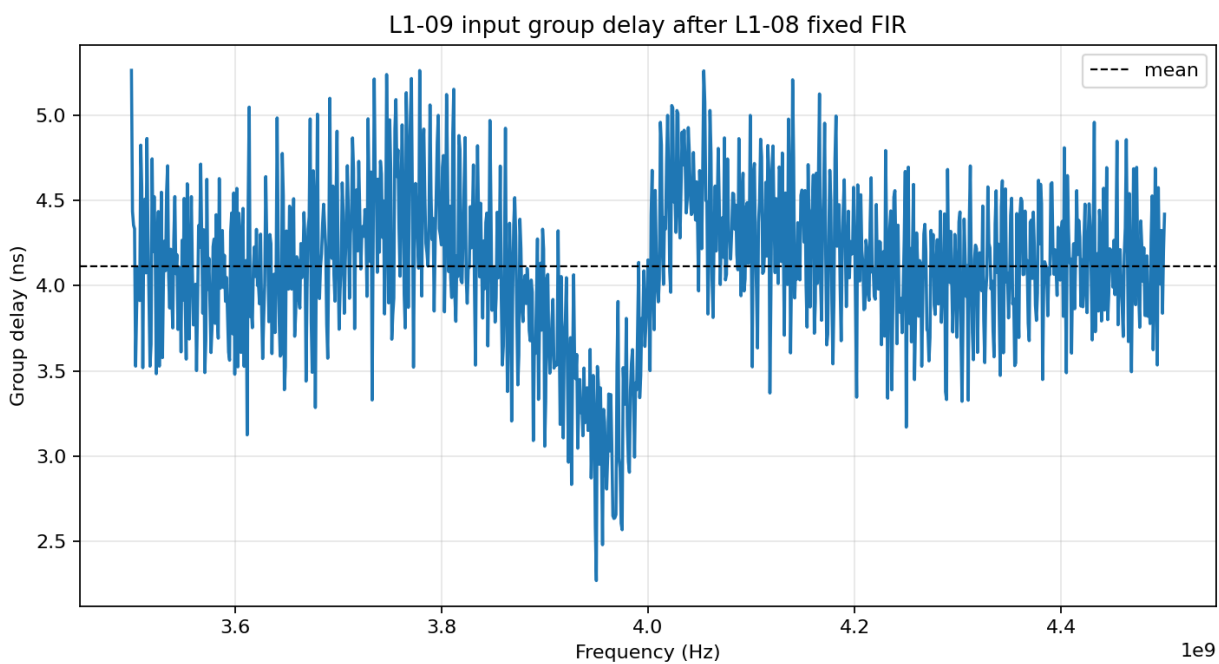


图 1: L1-09 group delay before compensation

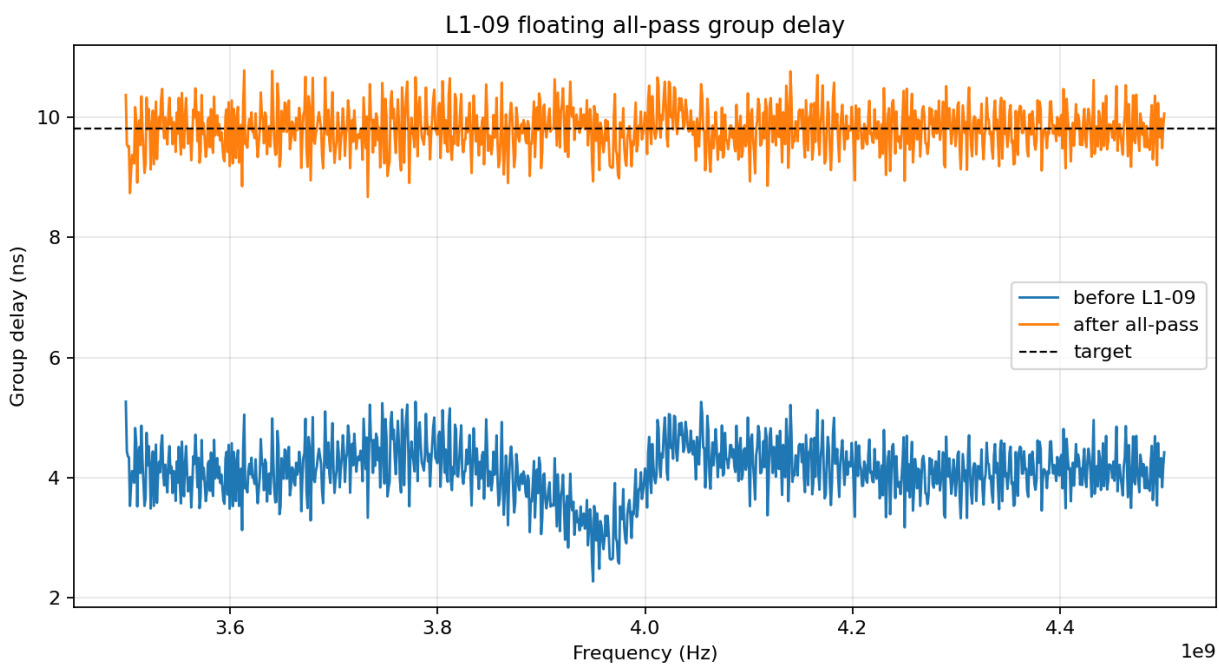


图 2: L1-09 group delay before and after all-pass compensation

$$\text{LSB} = 2^{-15} = 0.000030517578125$$

可表示范围约为:

$$[-4.0, 3.999969482422]$$

6.2 软件中如何模拟 bit 精度

当前软件模拟 fixed-point 的方式是:

`float coefficient`

↓

除以 `LSB`

↓

`round` 到最近整数

↓

`clip` 到 `fixed-point` 可表示整数范围

↓

乘回 `LSB`, 得到量化后的 `coefficient`

用公式表示:

$$\begin{aligned} q_{\text{int}} &= \text{round}\left(\frac{x}{\text{LSB}}\right), \\ q_{\text{int,clipped}} &= \text{clip}(q_{\text{int}}, \text{min_int}, \text{max_int}), \\ x_{\text{fixed}} &= q_{\text{int,clipped}} \cdot \text{LSB}. \end{aligned}$$

其中:

$$\text{LSB} = 2^{-\text{frac_bits}}$$

当前这一步模拟的是“系数存储精度”，也就是 all-pass filter 的浮点系数如果被写进硬件寄存器，需要用有限 bit 表示时会发生什么误差。

6.3 稳定性检查

量化后，程序重新用 fixed coefficients 组成 SOS，并计算每一级 denominator 的 pole。判断条件为:

$$\text{max_pole_radius} < 1 \Rightarrow \text{stable} = \text{True}$$

当前 clean run 中:

指标	数值
total bits	18
fraction bits	15
coefficient LSB	3.0517578125e-05
saturation count	0
max abs coefficient error	1.520833e-05
RMS coefficient error	8.819765e-06
max pole radius	0.954581
stable	True

这说明当前 Q3.15 系数量化后没有发生 saturation, pole 仍在单位圆内, all-pass IIR 数值稳定。

6.4 Fixed-Point 后的 Group Delay 影响

当前 fixed-point 量化后, group delay ripple 与 floating-point 结果几乎一致:

指标	数值
original group delay ripple	2.994724 ns
float compensated group delay ripple	2.108406 ns
fixed compensated group delay ripple	2.108794 ns
fixed vs float compensated GD RMS error	0.000353 ns
fixed vs float compensated GD max error	0.000869 ns

因此在当前 Q3.15 系数格式下, 系数量化本身没有明显破坏 L1-09 的补偿效果。

6.5 当前 Fixed-Point 仿真的限制

当前 fixed-point 仿真还不是完整 RTL 级 fixed-point。完整 RTL 级 fixed-point 还需要继续模拟:

input I/Q quantization

↓

每一级 all-pass filter 的乘法 fixed-point

↓

加法器 / accumulator fixed-point

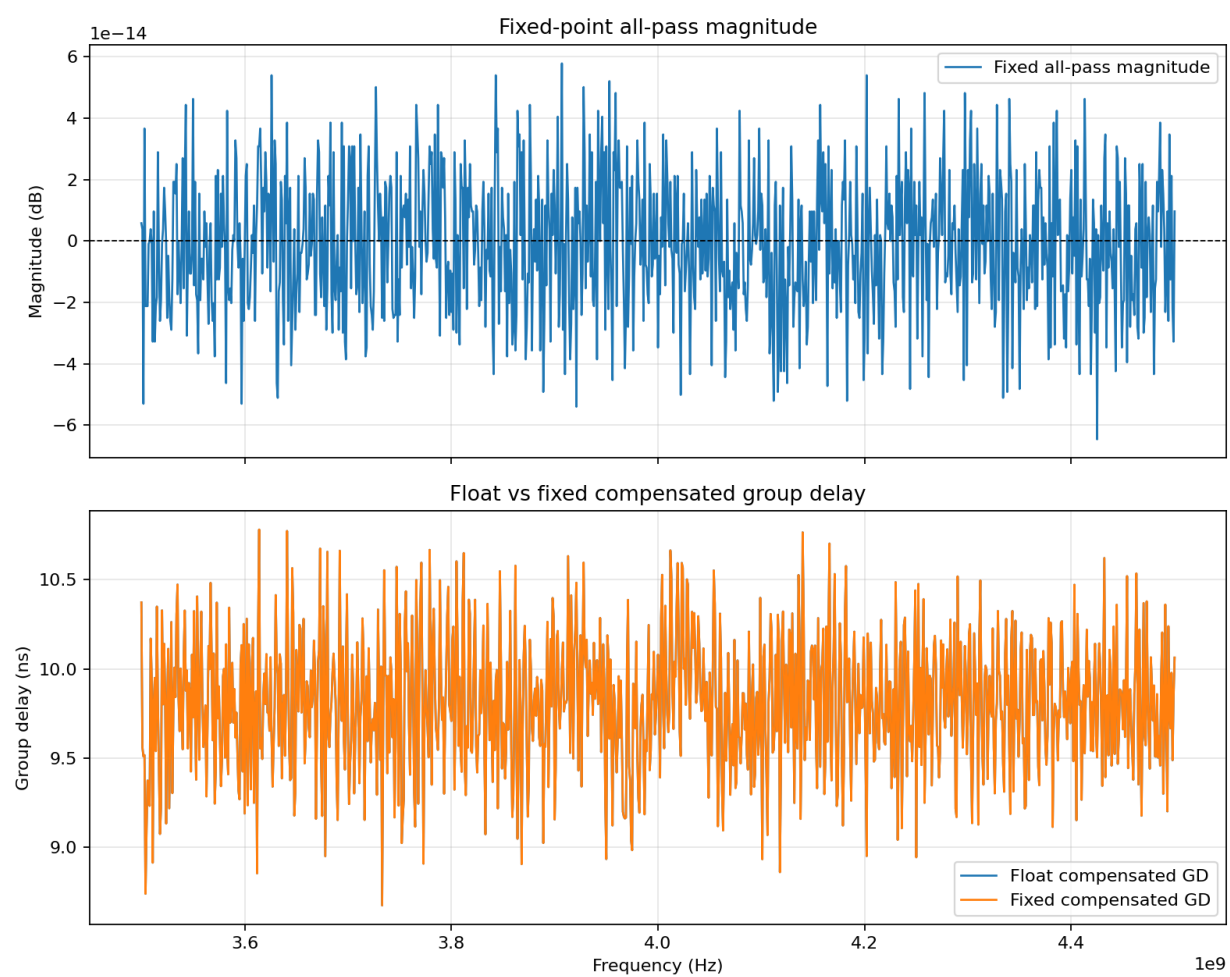


图 3: L1-09 fixed-point all-pass quantization

↓
 内部 delay register fixed-point
 ↓
 每一级输出 rounding / truncation / saturation
 ↓
 多级 SOS cascade 后的最终输出

当前阶段先验证了“系数量化后频率响应是否稳定、是否仍有补偿效果”。后续进入 RTL 设计时，需要把上述完整运算链条补进仿真模型。

7. EVM 验证方法

7.1 EVM_LIN

EVM_LIN 是基于频率响应的线性辅助指标。它不生成完整 QAM 时域波形，而是在频率响应上直接比较补偿前后相对于理想线性响应的误差。

当前程序会对以下三个 stage 计算 EVM_LIN：

```
after_h1
after_l1_08_fixed_fir
after_l1_08_fixed_fir_plus_l1_09_allpass
```

同时分解为：

```
evm_lin_percent
magnitude_only_evm_percent
phase_only_evm_percent
```

这种分解可以帮助判断误差主要来自幅度还是相位。

7.2 QAM EVM

QAM EVM 是更接近通信信号的验证方式。它会生成 QAM-loaded IF 信号，经过 H1、L1-08 fixed FIR 和 L1-09 all-pass 后，比较输出星座点相对于参考星座点的误差。

当前 QAM 配置为：

```
qam_order = 64
samples = 65536
freq_min_hz = 3.55 GHz
freq_max_hz = 4.45 GHz
qam_seed = 703592441
```


QAM EVM 更接近系统行为，但也更受信号构造、频率 bin、fitted delay 和 gain alignment 影响。

8. 当前实验结果

8.1 Clean Run 设置

本次报告引用的 clean run 为：

data/full_combined_20260605_155348
graph/full_combined_20260605_155348

主要 seed 为：

Seed 类型	数值
H1 seed	911204371
behavior seed	1401186327
QAM seed	703592441

L1-09 active 参数为：

参数	数值
all-pass sections	8
smooth window	31
margin ns	automatic
coefficient total bits	18
coefficient fractional bits	15

8.2 Group Delay 补偿结果

指标	数值
L1-09 输入 group delay mean	4.113093 ns
L1-09 输入 group delay ripple	2.994724 ns
floating all-pass 后 group delay ripple	2.108406 ns
fixed all-pass 后 group delay ripple	2.108794 ns

group delay ripple 降低量为:

$$2.994724 \text{ ns} - 2.108794 \text{ ns} = 0.885930 \text{ ns}$$

相对改善约为:

$$\frac{0.885930}{2.994724} = 29.6\%$$

因此当前 L1-09 确实降低了 group delay ripple, 但补偿后仍有约 2.109 ns 的 residual ripple, 说明当前 8-section all-pass 只能部分拟合该 seed 下的 group delay distortion。

8.3 EVM_LIN 结果

使用 fixed all-pass coefficients 时, EVM_LIN 结果为:

Stage	EVM_LIN	Magnitude-only EVM	Phase-only EVM
after H1	11.614517%	1.769356%	11.374248%
after L1-08 fixed FIR	11.433072%	0.695221%	11.373721%
after L1-08 fixed FIR + L1-09 all-pass	1.185585%	0.242099%	1.160319%

该结果说明:

1. L1-08 主要降低 magnitude-only EVM。
2. L1-09 主要降低 phase-only EVM。
3. 加入 L1-09 后, EVM_LIN 从 11.433072% 降到 1.185585%, 改善明显。

8.4 QAM EVM 结果

使用 fixed all-pass coefficients 时, QAM EVM 结果为:

Stage	QAM EVM	Magnitude-only EVM
after H1	11.733268%	1.633123%
after L1-08 fixed FIR	11.551006%	0.232999%
after L1-08 fixed FIR + L1-09 all-pass	3.396713%	2.263454%

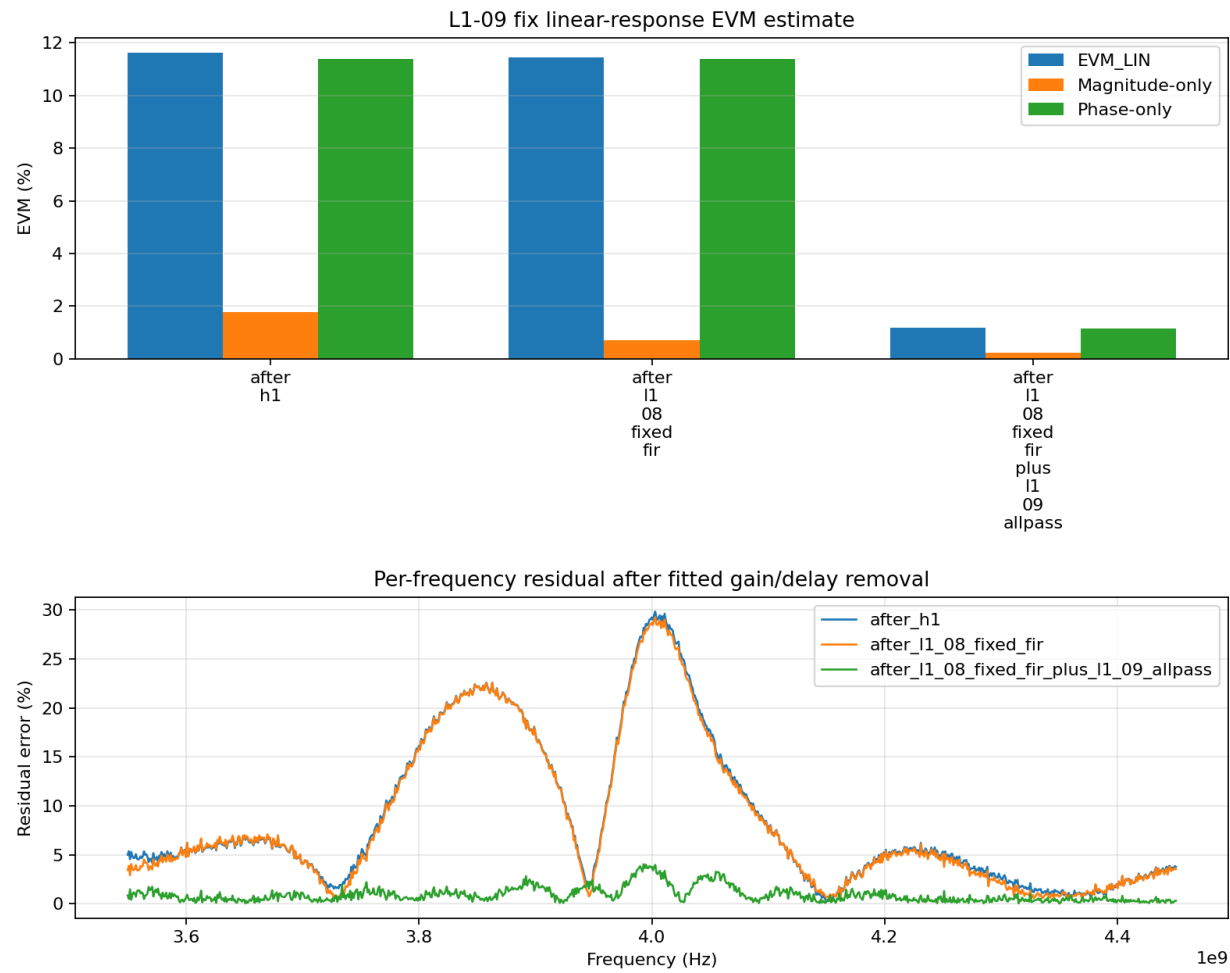


图 4: L1-09 fixed EVM_LIN

QAM EVM 从 11.551006% 降到 3.396713%，说明 L1-09 对相位相关失真有明显改善。但是 magnitude-only EVM 在加入 L1-09 后变大，这需要谨慎解释。

理论上 all-pass filter 幅度恒为 1，不应该改变 magnitude response。当前 QAM magnitude-only EVM 变大的原因可能是 All pass filter 初始化的 past output 如 $y[n-1]$ ($n = 0$) 时设置成为 0，也可能是其他原因，后续会意义判断

因此本报告对 QAM 结果的解读是：L1-09 明显降低整体 QAM EVM，但 QAM magnitude-only 分量的变化还需要后续进一步验证。

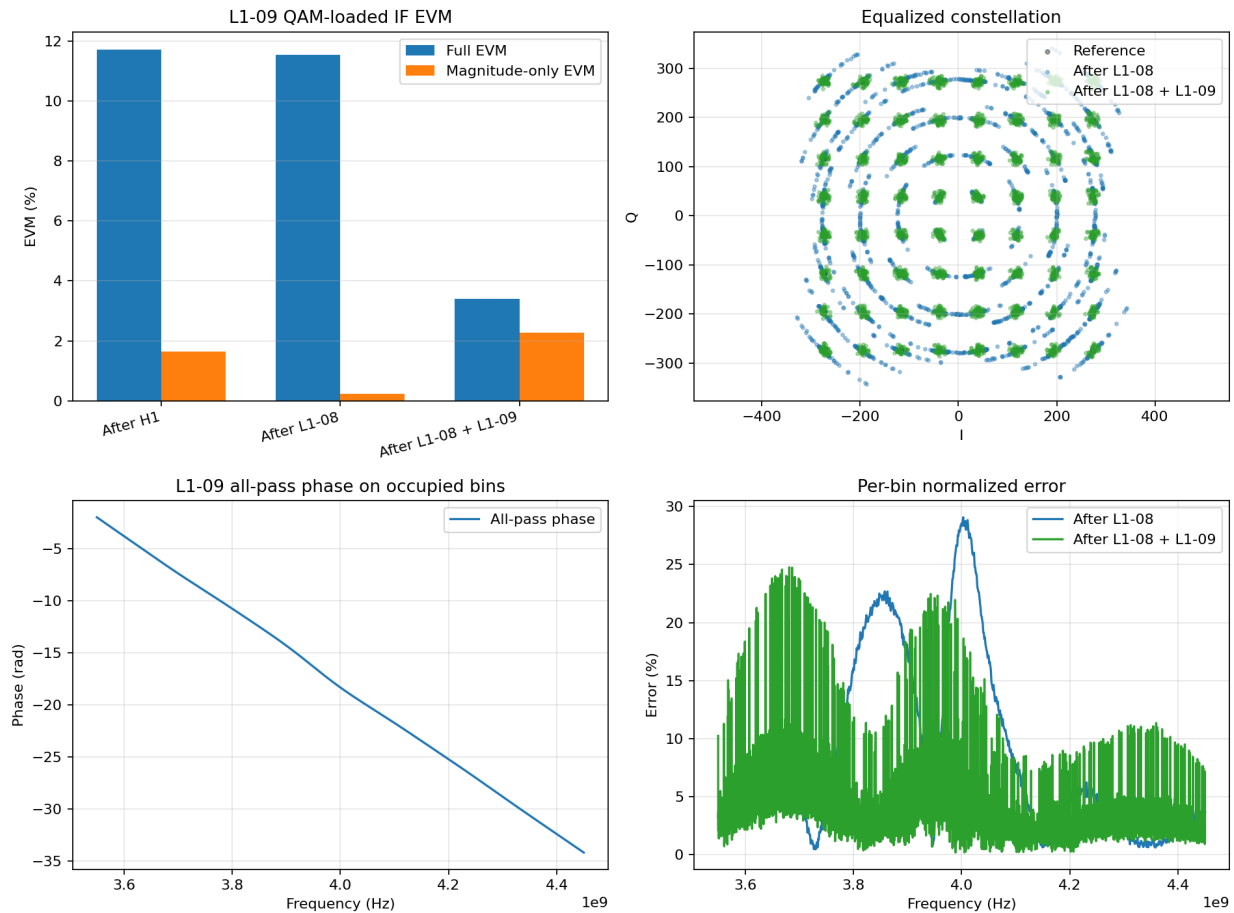


图 5: L1-09 fixed QAM EVM

9. 结论

当前 L1-09 行为级仿真已经完成从 group delay 分析、floating all-pass 设计、fixed-point 系数量化，到 EVM_LIN / QAM EVM 验证的完整流程。

主要结论如下：

1. L1-09 当前分析对象是 $H1 * H2_fixed$ ，也就是经过 L1-08 fixed-point FIR 后进入 L1-09 之前的响应。
2. 当前 8-section 二阶 all-pass IIR 可以降低 group delay ripple，但不能完全拉平。
3. 本次 clean run 中，group delay ripple 从 2.994724 ns 降到 fixed-point 后的 2.108794 ns。
4. fixed-point Q3.15 系数量化没有造成 coefficient saturation。
5. 量化后 max pole radius 为 0.954581，所有 pole 均位于单位圆内，因此 all-pass IIR 稳定。
6. EVM_LIN 从 11.433072% 降到 1.185585%，说明 L1-09 对 phase-only distortion 的补偿效果明显。
7. QAM EVM 从 11.551006% 降到 3.396713%，说明行为级通信信号验证也显示整体改善。
8. 当前 fixed-point 仍然是系数量化级别，不是完整 RTL 级 fixed-point 运算仿真。

综合来看，L1-09 当前算法方向是合理的，已经可以作为后续 sweep、RTL fixed-point 建模和硬件实现讨论的基础。

10. 后续工作

后续建议按以下顺序继续：

1. 增加 L1-09 sweep，覆盖 all-pass section count、fixed-point format、seed case 和 bandwidth profile。
2. 对比 6、8、10 个 all-pass section 在不同 H1 phase distortion 下的稳定性和补偿效果。
3. 当前 QAM magnitude-only 经过 L1-08+L1-09 增大的结果。
4. 建立完整 RTL 级 fixed-point 仿真，包括 I/Q input quantization、乘法器、加法器、accumulator、delay register 和逐级 rounding / saturation。
5. 将 L1-08 和 L1-09 的指标统一汇总，包括 magnitude ripple、group delay ripple、EVM_LIN、QAM EVM 和 fixed-point stability。